

**Salma Mahmoud** · You

Computer Science Student at October 6 Univer...
now ·



Loops in C#: The Art of Making Code Repeat Itself

من الحاجات اللي بدأت أفهمها بشكل أوضح وأنا بتعلم #C إن ساعات المشكلة مش في إزاي أعمل حاجة مرة واحدة...

المشكلة الحقيقية بتكون:

إزاي أخلي الكود يعمل نفس الحاجة كذا مرة من غير ما أكتب نفس السطور كل مرة؟

وهنا بييجي دور ال Loops.

بدل ما أكتب:

```
Console.WriteLine(1);
Console.WriteLine(2);
Console.WriteLine(3);
Console.WriteLine(4);
Console.WriteLine(5);
```

أقدر أستخدم:

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```



أبسط وأنصف بكثير

لكن وأنا بتعلم ال loops، اكتشفت إن الموضوع مش مجرد حفظ ال syntax.

الأهم هو إنني أعرف:

إمتي أستخدم كل نوع؟



1. for Loop — لما أكون عارفة العدد

أنا بفكر في for بالشكل ده:

“أنا عارفة عايزة أكرر الحاجة دي كام مرة.”

```
for (int i = 0; i < 5; i++)
```

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine("Hello C#");
}
```

الـ for فيها 3 أجزاء أساسية:

- * Initialization → أبدأ منين
- * Condition → إمتى أقف
- * Increment / Decrement → أتحرك إزاي

◆ 2. while Loop — Condition لما الـ

مع while مش لازم أكون عارفة عدد مرات التكرار من البداية.

المهم إن الـ condition لسه true.

```
int number = 1;
while (number <= 5)
{
    Console.WriteLine(number);
    number++;
}
```

هنا الـ condition بتتراجع قبل كل iteration.

وده خلاني أخذ بالي من حاجة مهمة:

⚠️ لازم أتأكد إن الـ condition هتبقى false في وقت ما.

لأن غير كده...

Hello, Infinite Loop 🙌😭

◆ 3. do-while — "نقذ الأول وبعدين اسأل"

الفرق هنا إن الكود بيتنفذ مرة واحدة على الأقل حتى لو الـ condition false. من البداية

```
int number = 1;
do
{
    Console.WriteLine(number);
}
```

```
{
    Console.WriteLine(number);
    number++;
}
while (number <= 5);
```

يعني ببساطة:

while → Check, then execute

do-while → Execute, then check

◆ 4. foreach — لما أتعامل مع Collections

لما أتعامل مع Array أو List، ال foreach بتكون أسهل.

```
string[] names = { "Salma", "Mona", "Sara" };
foreach (string name in names)
{
    Console.WriteLine(name);
}
```

مش محتاجة أتعامل مع ال index بنفسني.

ببساطة بقول:

"For each item in this collection, do this."

طيب أختار أنهي Loop؟ 

بدأت أفكر فيها بالطريقة دي:

for → iterations. لما أكون عارفة عدد ال

ومش شرط أعرف العدد من condition لما يكون عندي → while البداية.

do-while → لما أحتاج الكود يتنفذ مرة واحدة على الأقل.

foreach → Collection. لما أكون عايزة أعدّي على عناصر

وفي كمان:

break → loop يوقف الـ

continue → loop بعدها current iteration يتخطى الـ

☁️ إيه اللي فهمته من الـ Loops؟

في البداية، الـ loops كانت بالنسبة لي مجرد topic جديد محتاجة أحفظه.

لكن مع الـ practice بدأت أشوفها بشكل مختلف.

الـ loops مش فكرتها بس إننا نكرر الكود.

هي بتخلينا نفكر بطريقة أكثر كفاءة.

بدل ما أقول للـ computer:

"اعمل ده... وبعدين اعمله تاني... وبعدين تاني..."

أنا بقول له:

"دي القاعدة، وإنك عارف تعمل إيه."

واعتقد إن دي من الحاجات اللي بتخليني أبدأ أكتب cleaner and smarter code

#CSharp #DotNet #Programming
#SoftwareDevelopment #LearnCSharp #Coding
#FullStackDevelopment

Show translation



Questions

Q1:

depends on the data type. For example, 0 for int, false for bool, and null for reference types.

Q2:

Array.Clone() creates a shallow copy of the entire array, while Array.Copy() copies a specified number of elements from one array to another.

Q3:

Length returns the total number of elements in the array, while GetLength(dimension) returns the number of elements in a specific dimension.

Q4:

Array.Copy() copies elements from one array to another, but if an error occurs, the destination array may be partially modified.

Array.ConstrainedCopy() ensures the copy is all-or-nothing: if it fails, the destination array remains unchanged.

Q5:

Because foreach is simple, readable, and safe for read-only operations since it directly accesses each element without needing indexes.

Q6:

because it ensures user input is valid and prevents errors or unexpected program behavior.

Q7:

By using nested loops and formatting each element with consistent spacing
ex: Console.WriteLine("{0,5}", array[i,j]);
so the rows and columns align properly.

Q8:

Use switch when you need to compare one variable against multiple specific values. It is usually cleaner and more readable than many if-else statements.

Q9:

Array.Sort() has a time complexity of $O(n \log n)$.

Q10:

for can be a little faster because it uses indexes directly, but the difference is very small.

part02 Q3:

If the user enters a value outside the range of 1 to 7, the program displays an error message indicating that the input is invalid.

Bouns

Q5:

Stack: Usually 1 MB per thread in .NET on Windows.

Heap: No fixed default size;

it grows dynamically depending on the application and available memory.

Considerations: Stack is faster but limited, while Heap is larger and managed by the Garbage Collector.

Q6:

Time complexity is the measure of how the running time of an algorithm increases as the input size increases.

It is expressed using Big O notation